

Personalized Financial Goal-Achievement and Income-Growth Platform for Teens and Young Adults

Project Proposal for UCS503P

Submitted to: Ms. Nisha

SubGroup: 3Q25

Thapar Institute of Engineering and Technology

Shreya Sharma — 1024170260

Hunardeep Kaur — 1024170279

Kirandeep Kaur — 1024170280

Bhoomi Mahna — 1024170282

Introduction/Problem Statement

Managing personal finances effectively is becoming increasingly important, particularly for students and young professionals who are beginning to handle income, expenses, savings, investments, loans, and financial goals independently. Despite having access to digital payment and budgeting tools, many individuals struggle to **maintain consistent saving habits, control unnecessary spending, remember upcoming bills, and plan their finances** according to long-term objectives. Financial decisions are often made without a clear understanding of how current spending and saving patterns affect future goals.

A major gap exists in the availability of a single, user-friendly platform that goes beyond basic expense tracking and **provides personalized financial guidance and motivation**. Existing solutions often focus mainly on recording transactions or displaying expenses, while users may still lack support in maintaining saving discipline, **identifying suitable opportunities to increase their income, receiving timely alerts, or understanding possible investment options**. There is also limited integration of motivational elements such as progress tracking, streaks, rewards, and personalized recommendations that can encourage users to consistently follow their financial plans.

Our project aims to bridge this gap by providing a **personalized financial planning platform** that helps users manage their money while staying motivated toward their financial goals. The system is designed to provide goal-oriented planning, savings and spending tracking, alerts, reminders, progress and streak monitoring, and personalized recommendations based on the user's financial profile, interests, and qualifications. It also introduces **reward-based motivation through milestone achievements and coupons**, along with recommendations for **suitable side-income opportunities** and potential financial options. In this way, the platform aims to make personal financial management more structured, engaging, and goal-oriented.

Objective

- To assess the level of **financial-management awareness** among 100 college students aged 18-25, focusing on budgeting, saving, investing, and personal financial planning.
- To identify the key gaps in financial planning among students by **analyzing their awareness** of budgeting, saving habits, financial goals, and sources of income.
- To develop a **personalized financial-goal planning system** that converts users' income, expenses, goals, and target timelines into actionable weekly, monthly, and yearly plans.
- To evaluate the relevance of **personalized side-hustle recommendations** by matching users' skills and interests with suitable income-generating opportunities from multiple platforms.
- To evaluate whether **gamification features such as saving streaks, milestones, rewards, and progressive financial guidance** improve users' motivation toward consistent saving and goal achievement.

Methodology

1. System Architecture and Authentication

- The system will follow a layered client–server architecture with a Java Spring Boot backend exposing RESTful APIs.
- The backend will follow Controller–Service–Repository separation to isolate API handling, business logic, and data access.
- JWT-based authentication, password hashing, input validation, authorization, and centralized exception handling will secure user accounts and protected APIs.
- PostgreSQL/MySQL will store structured user, financial, goal, reward, bill, and opportunity data, with appropriate relationships and database indexing for efficient retrieval.
- The backend will remain stateless, allowing multiple Spring Boot instances to serve requests if the system scales.

2. User Profiling and Financial Goal Planning

- A structured questionnaire will collect users' career aspirations, income, expenditure, savings, skills, interests, financial goals, and target timelines.
- The business-logic layer will process these inputs and calculate the required savings amount, dividing the target into yearly, monthly, and weekly objectives.
- Database transactions will ensure consistency when updating goals, savings progress, streaks, and milestones.
- APIs will provide users with their current progress and upcoming targets, while pagination will be used for large datasets.

3. Side-Hustle Data Pipeline and Recommendation

- Relevant side-hustle opportunities will be collected from suitable online platforms through compliant APIs or web-scraping processes.
- Since scraping can be time-consuming, it will be executed through asynchronous background jobs rather than during user requests.
- A scheduler will periodically trigger data collection and refresh operations.
- A message queue/background-job mechanism will decouple scraping from the main application.
- Collected data will be cleaned, normalized, deduplicated, and stored in the database.
- Opportunities will be indexed using attributes such as category, skills, location, compensation, experience, and time commitment.
- A filtering and ranking service will match these attributes with the user's profile.
- Redis caching with TTL-based expiration will be used for frequently requested recommendations and filters, reducing database load and API latency.
- Failed background tasks will support retry mechanisms to improve reliability.

4. Coupons and Rewards

- A rule-based reward system will track savings, goals, and streaks. Coupons will unlock at predefined milestones, with messages such as “7 days left to unlock.”
- React + Tailwind CSS will display progress and rewards. Spring Boot will handle reward rules, while PostgreSQL stores streaks and coupon status.
- Redis will cache frequently accessed progress data to reduce repeated database queries.
- The frontend will communicate with the backend through REST APIs.
- All reward calculations will remain server-side to prevent manipulation from the client side.

5. Bill Reminders

- Users can add recurring bills with their amount, category, frequency, and due date.
- The system will identify upcoming and missed payments and generate reminders.
- Spring Boot will manage reminder logic, while scheduled background jobs will periodically check upcoming and overdue bills.
- PostgreSQL will store bill and payment records.
- React + Tailwind CSS will show upcoming, due, and overdue bills.
- Redis will provide faster access to frequently requested reminder data.
- Reminder processing will be handled asynchronously where notifications do not need to block the primary API request.

6. Future Financial Recommendations

- The system will analyze spending, savings, goals, interests, qualifications, and experience.
- It will generate personalized saving, side-hustle, and financial recommendations.
- Spring Boot will apply explainable rule-based logic to generate recommendations.
- PostgreSQL will provide the required user data, while Redis can cache frequently generated recommendations.
- React components will present these recommendations clearly.
- Progressive recommendations can be unlocked when users satisfy predefined financial milestones.

7. Scalability, Security and Performance

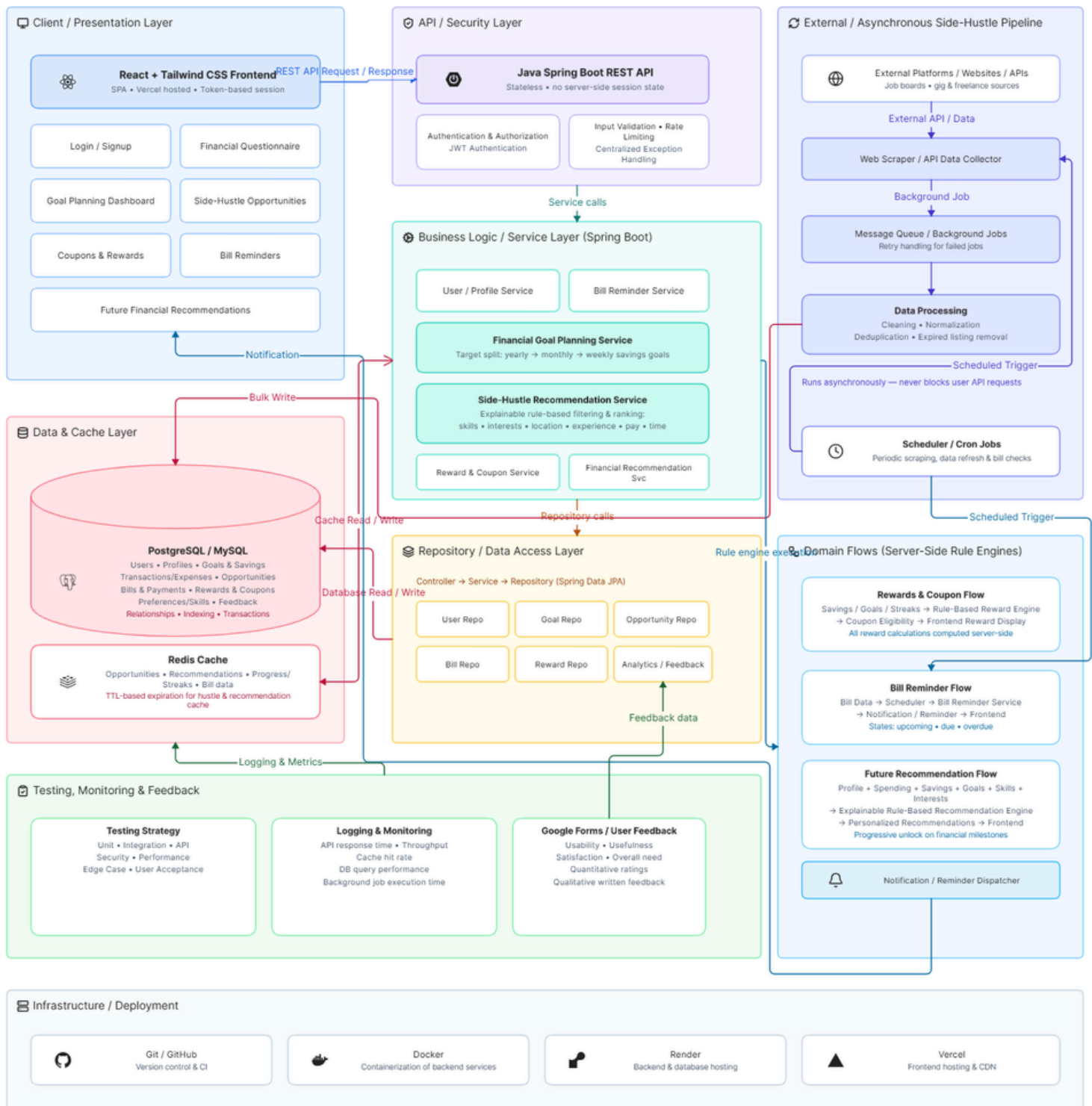
- Database indexing will improve retrieval of frequently filtered data such as users, opportunities, bills, and goals.
- Redis caching will reduce repeated database queries and improve API response time.
- Pagination will prevent large datasets from being returned in a single API response.
- Rate limiting will protect APIs from excessive requests and abuse.
- Logging and monitoring will identify API failures, slow requests, and background-job errors.
- The stateless Spring Boot architecture will support horizontal scaling by adding additional backend instances when required.
- Performance will be evaluated using API response time, throughput, cache-hit rate, database query performance, and background-job execution time.

8. Data Collection, Testing and Validation

- Testing will use sample user profiles with different income, spending, savings, and financial goals.
- Functional testing will cover financial planning, rewards, reminders, side-hustle recommendations, and future recommendations.
- The system will undergo unit testing, integration testing, API testing, security testing, and user acceptance testing.
- Individual business rules, authentication mechanisms, financial calculations, recommendation filters, and reward conditions will be tested independently.
- Integration testing will verify communication between the Spring Boot services, database, Redis cache, background workers, and external services.
- Edge cases including invalid financial inputs, expired opportunities, duplicate listings, failed external requests, concurrent updates, expired JWTs, and cache failures will be evaluated.

- A Google Form will collect ratings for usability, usefulness, satisfaction, and overall need. Quantitative responses will use ratings and percentages, while written feedback will be grouped into common themes.
- Git/GitHub will support collaboration. Vercel will host the frontend, while Render will host the backend and database.

Architecture Diagram



Timeline-:

WEEK	PHASE	KEY ACTIVITES	EXPECTED OUTCOME
WEEK 1	User Foundation	• Login/sign-up • User onboarding • Basic user profile	User account & financial profile
WEEK 2	Money Dashboard	• Weekly/monthly/yearly spending • Spending alerts & important updates	Functional financial dashboard
WEEK 3	Smart Opportunities	• Side-hustle recommendations • Personalized earning opportunities	Side-hustle recommendation system
WEEK 4	Saving Benefits	• Coupons, offers & discounts • Money-saving opportunities	Coupon & saving module
WEEK 5	SAGE Intelligence	• AI/chatbot integration • Financial guidance • Personalized goal suggestions	Smart financial assistan
WEEK 6	Goal & Bill Reminders	• Future financial goals • Bill reminders • Saving reminders	Reminder & goal system
WEEK 7	Testing & Refinement	• Bug fixing • UI improvements • Performance checks	Stable working prototype
WEEK 8	Buffer & User Testing	• Final user testing • Feedback collection • Final improvements	Final validated SAGE prototype

Resources / Budget

Team Roles

- Hunardeep Kaur, Kirandeep Kaur — Frontend development and UI/UX, building the interface, onboarding flow, and dashboard using React and Tailwind CSS.
- Bhoomi Mahna, Shreya Sharma — Backend development and database design, implementing REST APIs and core logic (plan generation, budget reallocation) using Java, Spring Boot, and PostgreSQL.

Material Resources

- Frontend: React and Tailwind CSS for a responsive, component-based UI.
- Backend: Java with Spring Boot for REST APIs, PostgreSQL for relational data storage.
- Development Tools: Visual Studio Code as the editor, Git and GitHub for version control.
- Deployment: Render for backend and database hosting, Vercel for frontend hosting.

Budget

- No external funding is required. All tools and frameworks used are open-source, and deployment relies entirely on free-tier hosting, sufficient for the project's demonstration and evaluation requirements.

Risks and Mitigations

Data privacy & security

Financial data may be exposed. Mitigate using JWT authentication, password hashing, input validation, authorization, and minimal data storage.

Incorrect recommendations

Suggestions may not fit user needs. Use explainable, rule-based logic based on user goals, skills, interests, and financial behavior.

Outdated data

Side-hustle listings and reminders may become stale. Use scheduled updates, validation, background processing, and retry mechanisms.

Performance & scalability

High request load may slow the system. Use Redis caching, indexing, pagination, and a stateless backend.

System & calculation errors

Incorrect computations may affect trust. Ensure reliability through unit, integration, API, security, and UAT testing, including edge cases.

Summary

- The Personalized Financial Goal-Achievement and Income-Growth Platform helps teenagers and young adults manage finances through goal tracking, expense monitoring, savings streaks, rewards, bill reminders, side-hustle discovery, and personalized recommendations.
- It uses a React + Tailwind frontend, Spring Boot backend, PostgreSQL database, and Redis caching, with a modular REST-based architecture for scalability and maintainability.
- The system will be validated through functional, integration, API, security, and user-acceptance testing, along with Google Form feedback to measure usability, satisfaction, and overall effectiveness.